

HOW NEGATIVE NUMBERS ARE STORED IN MEMORY

Integers ke negative no's ko store krne se shuru krte ha.

Example: **byte b = - 42 ;**

STEP-1] Sabse phle hum **Positive 42** ko binary m likhege jo hota ha **[00101010]** or positive value ase hi memory me store hoti ha. Or isko nikalne ki trick ha ki hume 42 ka **LCM** lena hoga or fir uske jo remainders ase hoge **[101010]** bs inko once place se lagao or jo space bche usme **0's** lga do jitni bits ka space bcha ha. Jse byte me abhi 2 space bche ha.

STEP-2] JAVA me negative no's ko save krne ke tarika ko **[2's COMPLIMENT]** khte ha.

1st COMPLIMENT : Isme hum particular no ke binary bits ko reverse kr dete ha. Matlab hum 1 ko 0 me, or 0 ko 1 me badal dege.

To 42 ke binary bit = [00101010]----->[11010101] This is our 1st Compliment.

2nd COMPLIMENT : Isko nikalne ke lia muj bs 1st Compliment me **1 ADD** krna ha.

To fir $11010101 + 1 =$ **11010110** , hence **Ye ha hmari final binary value jo memory me store hogi.**

Ek **important** baat jabhum memory se value ko reterive krege to kse pta chlega ki **11010101** value **+ve** no ki ha ya **-ve** no ki ? yha pr ate ha **MSB** and **LSB**.

MSB [Most Significant Bit]: MSB kisi bhi number ki left most side hoti ha, or iski significant value zada hoti ha.

Ex: 259874, to yha pr 2 MSB ha.

LSB [Least Significant Bit]: LSB kisi bhi number ki right most side hoti ha, or iski significant value km hoti ha.

Ex: 259874, to yha pr 4 LSB ha.

To **CONCLUSION** ye ha agar kisi number ki **MSB** value **1** ha to vo **-ve** no ha, or agar **MSB** value **0** ha to o **+ve** no ha.

HOW FLOATING NUMBERS ARE STORED IN MEMORY

FLOATING NUMBERS
Total Bits=32 , they are divided in 3 parts.
1Bit=Sign Bit, 8 Bits=Exponent, 23Bits= Mantissa
float f = 8.125f ;
SAVING TO MEMORY
STEP 1) firstly convert 8 to direct binary language= 1000 by doing LCM.
STEP 2) Now, convert 0.125 to binary= Iske liye hume 0.125 ko 2 se multiply krna pdega, So, $0.125 * 2 = 0.25$ $0.25 * 2 = 0.5$ $0.5 * 2 = 1.0$ ∴ iska binary code bna 001 , teeno result ke 1st digits. Now binary value of 8.125 is 1000.001
STEP 3)

Make it in form of $(1.xx) \times 2^{\text{exponent}} \Rightarrow 1.000001 \times 2^3$

∴ humne 1000.001 ko $1.xx$ me convert krne k lia decimal ko three digit aage laya. $\Rightarrow 1.000001$ or decimal jitne digit aage aya utni value +sign ke sath 2 ke exp. Me jud jaygi.

STEP 4)

Add **BIAS** to the exponent=3

BIAS value=127

$\Rightarrow 3+127=130$

Now, convert 130 into binary value by doing LCM.=**10000010**, ye value 8 BIT **exponent** me aayegi.

STEP 5)

Place values in **MEMORY**=Total=32 bits

1 Sign bit = 0 [kuki 8.125 +ve value ha]

8 Exponent = 10000010 [ye step-4 se mili ha.]

23 Mantissa = 000001_000000000000000000 [ye value xx wali ha jo hume Step-3 se mili ha] or jo blank spaces ha usme 0's lg jaygi jab tak 23 bits complete nhi hogi

RETERIVE FROM MEMORY

Print Statement $\Rightarrow$ System.out.println(f);

Formula= $(-1)^{\text{SIGN BIT}} * (1 + \text{Mentissa}) * 2^{\text{exponent} - \text{BIAS}}$

$\Rightarrow (-1)^0 * (1 + \text{Mentissa}) * 2^{130-127}$

= $0 * (1 + \text{Mentissa}) * 2^3$

$\Rightarrow$ **mentissa = 2^{-6}** kuki mentissa decimal ke baad ki value ha to isko $2^{-1}, 2^{-2}, 2^{-3}, \dots$ me left to right count krege jaha-jaha hume '1' milega. To yha hmari value ati ha 2^{-6}

= $0 * (1 + 2^{-6}) * 8$, or isko solve krke aa jaygi value= **8.125** ans.

BIAS

Float vaulue ko memory me store krna ka jo 32bit format ha ye **IEEE STANDARD** ke hisab se ha.

Or **IEEE** ne **BIAS** ko sirf is liye banaya taki koi bhi **exponent** value -ve na rhe.

Formula= $(2^{(\text{exponent}-1)}-1) \Rightarrow 2^{8-1}-1 = 2^7-1 = 128-1 = 127, \text{BIAS value.}$

HOW DOUBLE NUMBERS ARE STORED IN MEMORY

There is a same concept with **double** numbers.

Bs double me bits **64** ho jati ha or exponent bhii **11** ho jate ha.

Baki sara process same hi rhta ha.

Or double me **BIAS 1023** hota ha.

S	M	T	W	T	F	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

27

027/08 • WK. 05

TUESDAY

JANUARY

Floating value ko memory me store krne ki trick

Ques-1) float $f = 8.125$;

Sol. → Step-1 Convert 8 & 0.125 into binary value

$$\therefore 8 = 1000 \quad \& \quad 0.125 = 001$$

2	8	
2	4	0
2	2	0
1	1	0

$$0.125 \times 2 = 0.25 \rightarrow 0$$

$$0.25 \times 2 = 0.5 \rightarrow 0$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

Read Top to down

$$\therefore 8.125 = 1000.001 \text{ Binary value}$$

Step-2 Make it in the form of $(1.xxx) \times 2^{\text{Exp}}$

$$\text{So } 1000.001 \Rightarrow (1.000001) \times 2^3$$

★ Reason humne (.) ko teen digit aare layer to 2 ki power 3 agiya
 ye $(1.xxx) \times 2^{\text{Exp}}$ ki form me aa gya.

Step-3 Add BIAS to Exponent.

for float BIAS = 127

$$\Rightarrow 3 + 127 = 130$$

ab 130 ka binary value: 10000010 ye exponent ka

Step-4 Place value in memory

Sign
1-Bit | 8 → Exponent | 23 → Mantissa

M	T	W	T	F	S	S
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

Sign Bit me 0 aage kuki 8.125 JANUARY
the value ha.

0 | 10000010 | 000001000000000000000000
(XXXX) wala part mantissa ha

Now, jab hum retrieve krge to kse hoga. `system.out.println()`

Formula → $(-1)^{\text{Sign}} \times (1 + \text{Mantissa}) \times 2^{\text{Exp-Bias}}$

$$= (-1)^0 \times (1 + 0000010) \times 2^{10000010 - 127}$$

$$0 \times (1 + \frac{-6}{2}) \times 2^{130 - 127}$$

$$0 \times (1 + \frac{-6}{2}) \times 2^3$$

$$0 \times (1 + \frac{1}{26}) \times 8$$

$$0 \times (1 + \frac{1}{64}) \times 8$$

$$0 \times (1 + 0.015625) \times 8$$

$$\Rightarrow (1.015625) \times 8$$

$$\Rightarrow \boxed{8.125 \text{ Ans}} \rightarrow \text{Value}$$

29

029-030 • WK 08

JANUARY

THURSDAY

How $\rightarrow 0.7$ is stored

0.7

~~0.7~~ 0.7 binary = 0

$$0.7 = 0.7 \times 2 = 1.4 = 1$$

$$0.4 \times 2 = 0.8 = 0$$

$$0.8 \times 2 = 1.6 = 1$$

$$0.6 \times 2 = 1.2 = 1$$

$$0.2 \times 2 = 0.4 = 0$$

$$0.4 \times 2 = 0.8 = 0$$

$$0.7 \Rightarrow 1.01100110110110 \dots$$

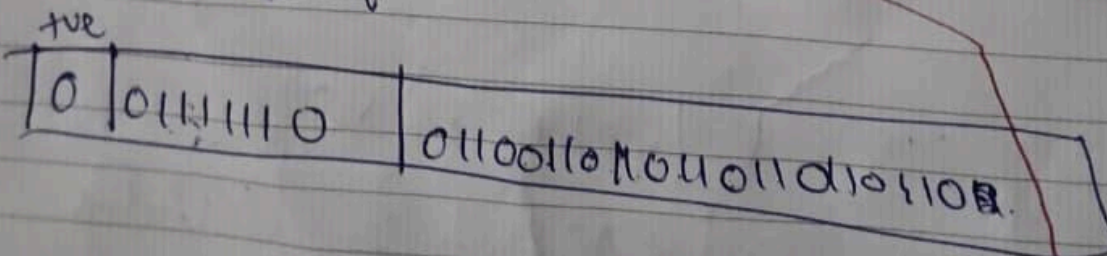
$$0.7 \Rightarrow 0.101100110110110 \dots$$

Now represent in form of $(1.xx)x_2$

$$0.101100110110110 \dots \Rightarrow (1.01100110110110 \dots) \times 2^{-1}$$

Now add BIAS $\Rightarrow -1 + 127 = 126$

Now Binary of 126 $\Rightarrow 01111110$



DECEMBER - 2014

S	M	T	W	T	F	S
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

$$0.7$$

$$\times 2$$

$$\hline 1.4$$

$$0.4$$

$$\times 2$$

$$\hline 0.8$$

$$\times 2$$

$$\hline 1.6$$

$$0.6$$

$$\times 2$$

$$\hline 1.2$$

$$0.2$$

$$\times 2$$

$$\hline 0.4$$

2	126
2	63
2	31
2	15
2	7
2	3
2	1

2015

M	T	W	T	F	S
2	3	4	5	6	7
9	10	11	12	13	14
16	17	18	19	20	21
23	24	25	26	27	28

$$\begin{array}{r} 64 \\ + 2 \\ \hline 128 \end{array}$$

JANUARY - 2015

	S	M	T	W	T	F	S
1							
2							
3							
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							
20							
21							
22							
23							
24							
25							
26							
27							
28							
29							
30							
31							

$x^2 + x + 2$

$(1) - 0 \times (1 + 2)$

$$(-1) - 0 \times (1 + 2^{-2}) \times 2$$

$$-0 \quad \left(1 + \frac{1}{2}\right) \times 2^{(-1)}$$

$$\Rightarrow \left(1 + \frac{1}{4}\right)^{2^2} \times (-2)^{\frac{1}{2}}$$

$$\begin{aligned} & (1 + 0.75) \times (-2) \\ & 1.75 \times (-2) \end{aligned}$$

12 Double →

$$\begin{array}{r} 1 \\ 11.25 \\ \times 2 \\ \hline 2.50 \end{array}$$

total = 64 Bits

